Divide and Conquer

SC51398: Design & Analysis of Algorithms

Krishnendu Jana

September 18, 2025

BCA Program
Department of Computer Science

A Taxonomy of Algorithmic Strategies

The Problem-Solving Toolbox

Most algorithms can be categorized by their core design strategy or **paradigm**. Understanding these paradigms gives you a toolbox for solving new problems.

- Brute-Force
- Greedy
- Divide and Conquer
- Dynamic Programming
- Backtracking
- Randomized & Approximation

The rest of this course is a deep dive into these powerful techniques.

Core Paradigms

Brute-Force
Idea: Just do it. Systematically try every single possibility. **Example:** Linear search (checking every element in a list one by one). **Use When:** The problem size is small, or as a baseline for comparison.

Core Paradigms

Brute-Force
Idea: Just do it. Systematically try every single possibility. **Example:** Linear search (checking every element in a list one by one). **Use When:** The problem size is small, or as a baseline for comparison.

Greedy Algorithms
Idea: Take the best you can get right now. Make the locally optimal choice at each step. **Example:** Making change with the fewest coins. **Use When:** The problem has the "greedy choice property" (proven to be optimal).

MIT Vishwaprayag University

Core Paradigms (Cont.)

Divide and Conquer
Idea: Divide the problem into smaller subproblems, solve them recursively, and combine the results. **Example:** Merge Sort, Quicksort, Binary Search. **Use When:** The problem can be naturally split into independent subproblems.

MIT Vishwaprayag University

Core Paradigms (Cont.)

Divide and Conquer
Idea: Divide the problem into smaller subproblems, solve them recursively, and combine the results. **Example:** Merge Sort, Quicksort, Binary Search. **Use When:** The problem can be naturally split into independent subproblems.

Dynamic Programming
Idea: Don't solve the same problem twice. Break down a problem into *overlapping* subproblems and store their results to avoid re-computation. **Example:** Calculating Fibonacci numbers efficiently. **Use When:** The problem has optimal substructure and overlapping subproblems.

MIT Vishwaprayag University

Core Paradigms (Cont.)

Backtracking
Idea: Explore, and if you hit a dead end, go back and try something else. **Example:** Solving a Sudoku puzzle or a maze. **Use When:** You need to find a solution through constraint satisfaction.

MIT Vishwaprayag University

Core Paradigms (Cont.)

Backtracking
Idea: Explore, and if you hit a dead end, go back and try something else. **Example:** Solving a Sudoku puzzle or a maze. **Use When:** You need to find a solution through constraint satisfaction.

Randomized & Approximation
Idea: Sacrifice a degree of certainty or optimality for a huge gain in speed.

- **Randomized:** Use randomness to achieve good performance on average (e.g., Randomized Quicksort).
- **Approximation:** Find a "good enough" solution for a hard problem where the perfect solution is too slow (e.g., Traveling Salesman Problem).

MIT Vishwaprayag University

Questions?

We have built the foundation. Next, we begin our deep dive into these powerful algorithmic strategies.

Divide and Conquer

The divide-and-conquer design paradigm

1. ***Divide*** the problem (instance) into subproblems.
2. ***Conquer*** the subproblems by solving them recursively.
3. ***Combine*** subproblem solutions.

Binary search

Find an element in a sorted array:

1. ***Divide:*** Check middle element.
2. ***Conquer:*** Recursively search 1 subarray.
3. ***Combine:*** Trivial.

Example: Find 9

3	5	7	8	9	12	15

Binary search

Find an element in a sorted array:

1. ***Divide:*** Check middle element.
2. ***Conquer:*** Recursively search 1 subarray.
3. ***Combine:*** Trivial.

Example: Find 9

3	5	7	8	9	12	15

Binary search

Find an element in a sorted array:

1. ***Divide:*** Check middle element.
2. ***Conquer:*** Recursively search 1 subarray.
3. ***Combine:*** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

1. ***Divide:*** Check middle element.
2. ***Conquer:*** Recursively search 1 subarray.
3. ***Combine:*** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

1. ***Divide:*** Check middle element.
2. ***Conquer:*** Recursively search 1 subarray.
3. ***Combine:*** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

1. ***Divide:*** Check middle element.
2. ***Conquer:*** Recursively search 1 subarray.
3. ***Combine:*** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Recurrence for binary search

$$T(n) = 1\,T(n/2) + \Theta(1)$$

 # subproblems

subproblem size

work dividing and combining

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow \text{CASE 2 } (k = 0)$$

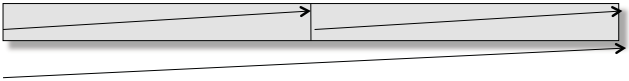
$$\Rightarrow T(n) = \Theta(\lg n)\ .$$

Meet Merge Sort

divide and conquer

MERGE-SORT $A[1 \,.\,.\, n]$

1. If $n = 1$, done (nothing to sort).
2. Otherwise, recursively sort $A[\,1 \,.\,.\, n/2\,]$ and $A[\,n/2+1 \,.\,.\, n\,]$.
3. "***Merge***" the two sorted sub-arrays.



Key subroutine: **MERGE**

Merging two sorted arrays

20	12
13	11
7	9
2	1

Merging two sorted arrays

20	12
13	11
7	9
2	1

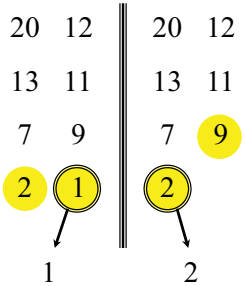
1

Merging two sorted arrays

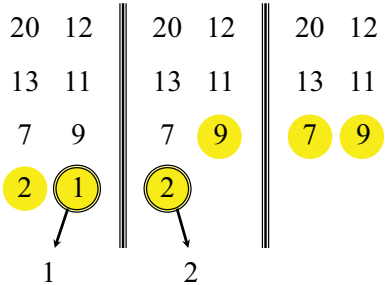
20	12	20	12
13	11	13	11
7	9	7	9
2	1	2	

1

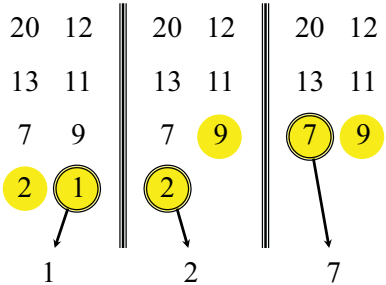
Merging two sorted arrays



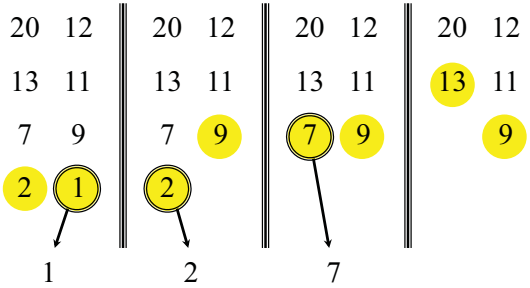
Merging two sorted arrays



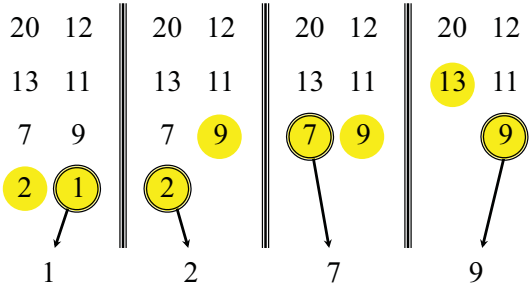
Merging two sorted arrays



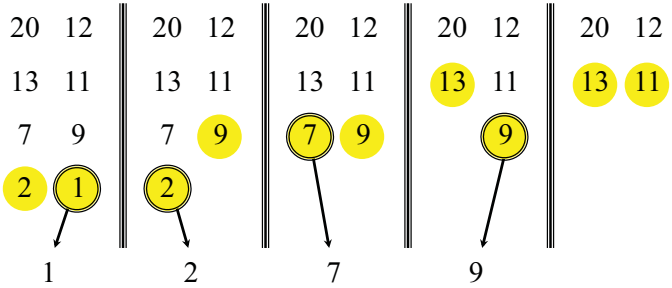
Merging two sorted arrays



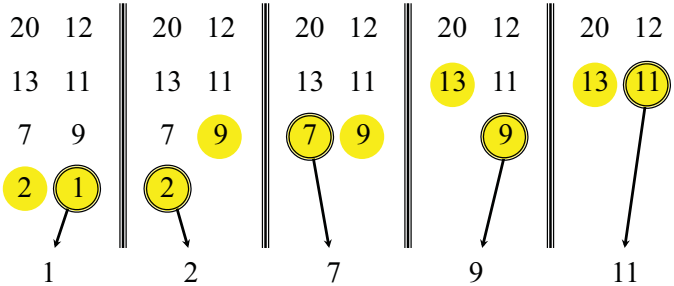
Merging two sorted arrays



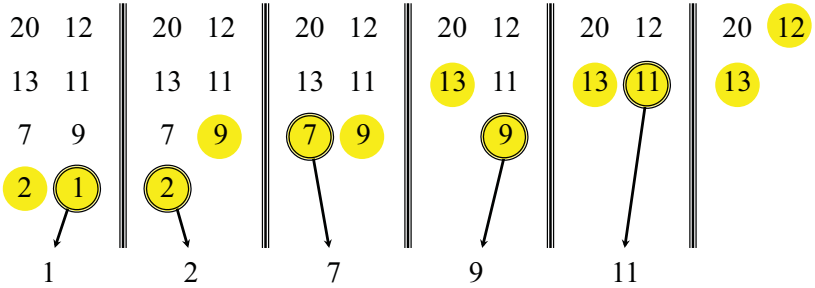
Merging two sorted arrays



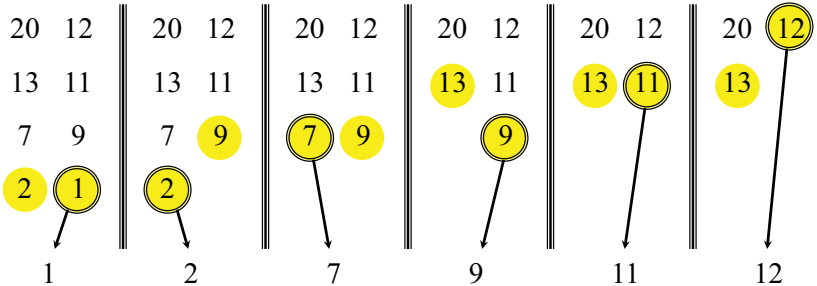
Merging two sorted arrays



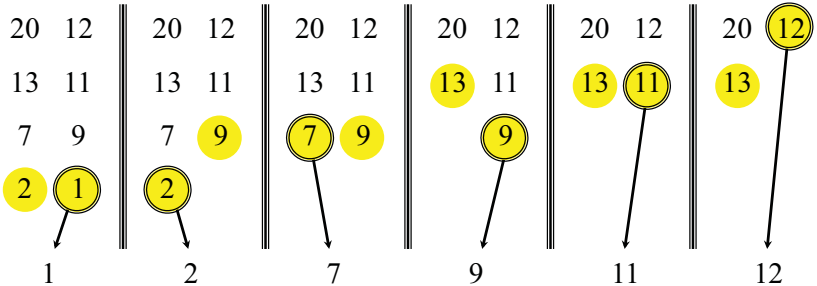
Merging two sorted arrays



Merging two sorted arrays



Merging two sorted arrays



Time = $\Theta(n)$ to merge a total of n elements (linear time).

Merging Two Sorted Sequences

- The conquer step of merge-sort consists of merging two sorted sequences A and B into a sorted sequence S containing the union of the elements of A and B
- Merging two sorted sequences, each with $n/2$ elements and implemented by means of a doubly linked list, takes $O(n)$ time

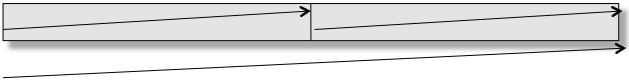
```
Algorithm merge(A, B)
    Input sequences A and B with
        n/2 elements each
    Output sorted sequence of A ∪ B

    S ← empty sequence
    while ¬A.isEmpty() ∧ ¬B.isEmpty()
        if A.first().element() < B.first().element()
            S.insertLast(A.remove(A.first()))
        else
            S.insertLast(B.remove(B.first()))
    while ¬A.isEmpty()
        S.insertLast(A.remove(A.first()))
    while ¬B.isEmpty()
        S.insertLast(B.remove(B.first()))
    return S
```

Meet Merge Sort

divide and conquer

MERGE-SORT $A[1 \,.\,.\, n]$

1. If $n = 1$, done (nothing to sort).
2. Otherwise, recursively sort $A[\,1 \,.\,.\, n/2\,]$ and $A[\,n/2+1 \,.\,.\, n\,]$.
3. "***Merge***" the two sorted sub-arrays.



Key subroutine: **MERGE**

Analyzing merge sort

MERGE-SORT $A[1 \,..\, n]$	$T(n)$
1. If $n = 1$, done.	$\Theta(1)$
2. Recursively sort $A[\,1 \,..\, \lceil n/2 \rceil\,]$ and $A[\,\lceil n/2 \rceil + 1 \,..\, n\,]$.	$2T(n/2)$
3. ***"Merge"*** the two sorted lists	$\Theta(n)$

$$T(n) = \begin{cases} \Theta(1) \text{ if } n = 1; \\ 2T(n/2) + \Theta(n) \text{ if } n > 1. \end{cases}$$

$$T(n) = ?$$

Recurrence solving

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

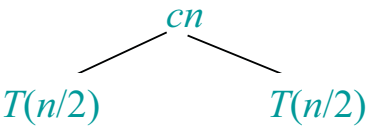
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

$$T(n)$$

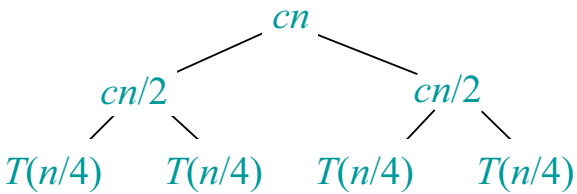
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



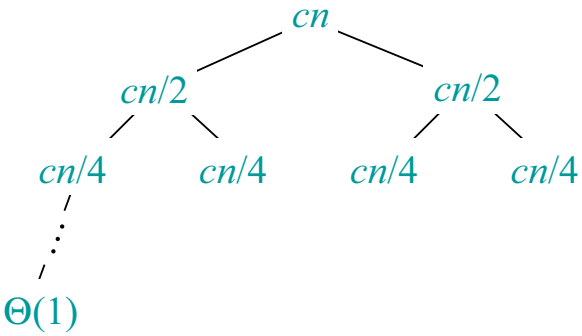
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



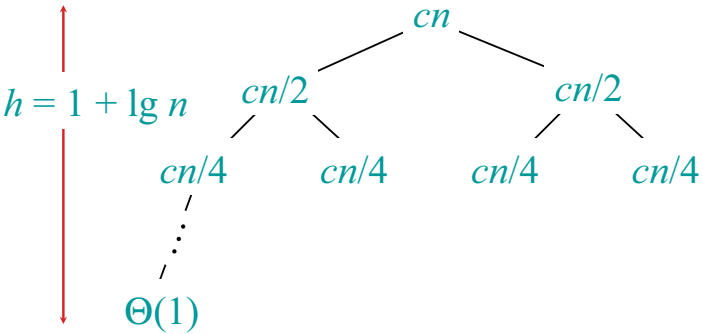
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



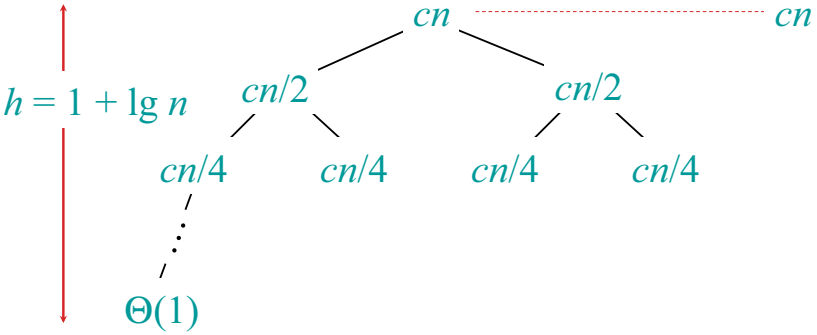
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



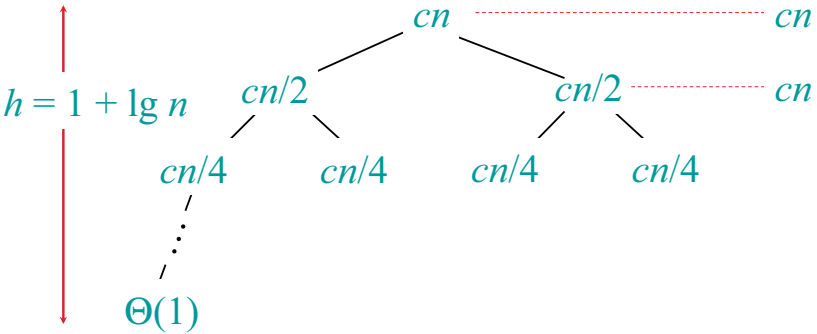
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



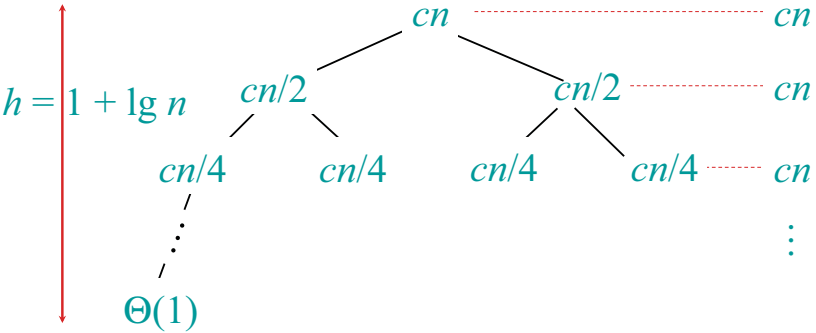
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



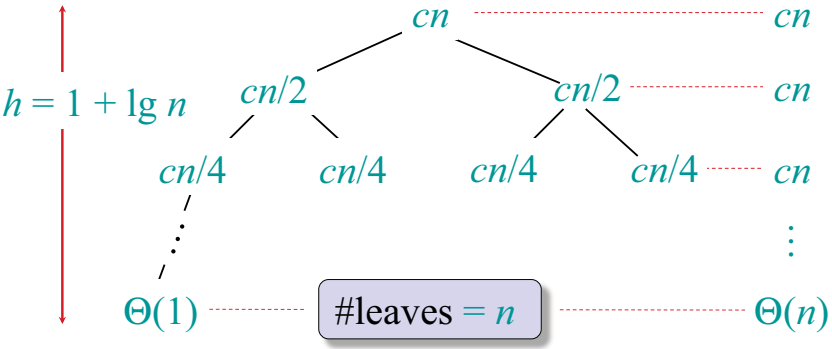
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



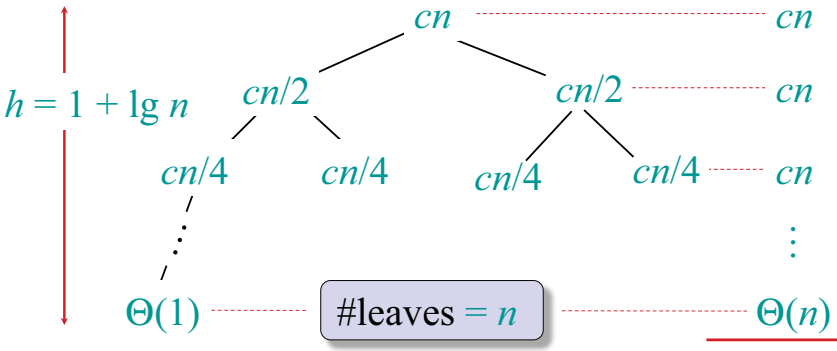
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Recursion tree

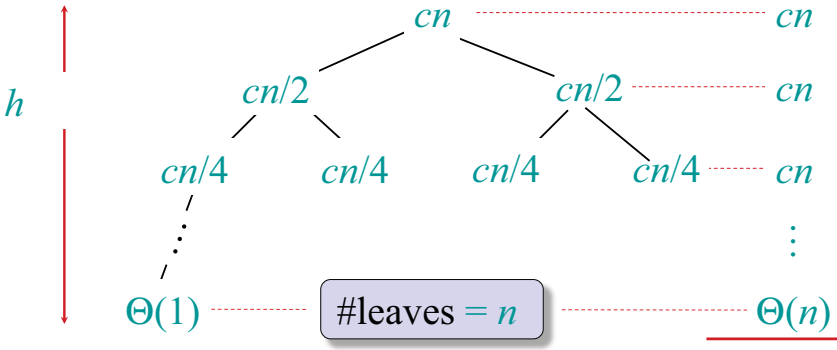
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Total ?

Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Total $= \Theta(n \lg n)$

Equal amount of work done at each level

Powering a number

Problem: Compute a^n, where $n \in \boldsymbol{N}$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if } n \text{ is odd.} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\lg n) .$$

Fibonacci numbers

Recursive definition:

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2. \end{cases}$$

$$0 \quad 1 \quad 1 \quad 2 \quad 3 \quad 5 \quad 8 \quad 13 \quad 21 \quad 34 \quad \cdots$$

Naive recursive algorithm: $\Omega(\phi^n)$ (exponential time), where $\phi = (1+\sqrt{5})/2$ is the ***golden ratio***.

Computing Fibonacci numbers

Naive recursive squaring:

$F_n = \phi^n/\sqrt{5}$ rounded to the nearest integer.

- Recursive squaring: $\Theta(\lg n)$ time.
- This method is unreliable, since floating-point arithmetic is prone to round-off errors.

Bottom-up:

- Compute $F_0, F_1, F_2, \ldots, F_n$ in order, forming each number by summing the two previous.
- Running time: $\Theta(n)$.

Recursive squaring

Theorem: $\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$.

Algorithm: Recursive squaring.
Time = $\Theta(\lg n)$.

Proof of theorem. (Induction on n.)

Base ($n = 1$): $\begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^1$.

Recursive squaring

Inductive step ($n \geq 2$):

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n} \qquad \blacksquare$$

Matrix multiplication

$$\left.\begin{array}{ll}\textbf{Input:} & A = [a_{ij}],\ B = [b_{ij}]. \\ \textbf{Output:} & C = [c_{ij}] = A \cdot B.\end{array}\right\} \quad i, j = 1, 2, \ldots, n.$$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

Standard algorithm

for $i \leftarrow 1$ **to** n
  do for $j \leftarrow 1$ **to** n
    do $c_{ij} \leftarrow 0$
      for $k \leftarrow 1$ **to** n
        do $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$

Running time $= \Theta(n^3)$

Divide-and-conquer algorithm

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\left[\begin{array}{c|c} r & s \\ \hline t & u \end{array}\right] = \left[\begin{array}{c|c} a & b \\ \hline c & d \end{array}\right] \cdot \left[\begin{array}{c|c} e & f \\ \hline g & h \end{array}\right]$$

$$C \quad = \quad A \quad \cdot \quad B$$

$$\left.\begin{aligned} r &= ae + bg \\ s &= af + bh \\ t &= ce + dh \\ u &= cf + dh \end{aligned}\right\} \begin{array}{l} 8 \text{ mults of } (n/2) \times (n/2) \text{ submatrices} \\ 4 \text{ adds of } (n/2) \times (n/2) \text{ submatrices} \end{array}$$

Analysis of D&C algorithm

$$T(n) = 8\,T(n/2) + \Theta(n^2)$$

 # submatrices

submatrix size

work adding submatrices

$$n^{\log_b a} = n^{\log_2 8} = n^3 \;\Rightarrow\; \text{CASE 1} \;\Rightarrow\; T(n) = \Theta(n^3).$$

No better than the ordinary algorithm.

Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$
$$P_2 = (a + b) \cdot h$$
$$P_3 = (c + d) \cdot e$$
$$P_4 = d \cdot (g - e)$$
$$P_5 = (a + d) \cdot (e + h)$$
$$P_6 = (b - d) \cdot (g + h)$$
$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$
$$s = P_1 + P_2$$
$$t = P_3 + P_4$$
$$u = P_5 + P_1 - P_3 - P_7$$

7 mults, 18 adds/subs.
Note: No reliance on commutativity of mult.

Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$
\begin{aligned}
P_1 &= a \cdot (f - h) \\
P_2 &= (a + b) \cdot h \\
P_3 &= (c + d) \cdot e \\
P_4 &= d \cdot (g - e) \\
P_5 &= (a + d) \cdot (e + h) \\
P_6 &= (b - d) \cdot (g + h) \\
P_7 &= (a - c) \cdot (e + f)
\end{aligned}
$$

$$
\begin{aligned}
r &= P_5 + P_4 - P_2 + P_6 \\
&= (a + d)(e + h) \\
&\quad + d(g - e) - (a + b)h \\
&\quad + (b - d)(g + h) \\
&= ae + ah + de + dh \\
&\quad + dg - de - ah - bh \\
&\quad + bg + bh - dg - dh \\
&= ae + bg
\end{aligned}
$$

Strassen's algorithm

1. ***Divide:*** Partition A and B into $(n/2)\times(n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
2. ***Conquer:*** Perform 7 multiplications of $(n/2)\times(n/2)$ submatrices recursively.
3. ***Combine:*** Form C using $+$ and $-$ on $(n/2)\times(n/2)$ submatrices.

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

Analysis of Strassen

$$T(n) = 7\,T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \;\Rightarrow\; \text{CASE 1} \;\Rightarrow\; T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 30$ or so.

Best to date (of theoretical interest only): $\Theta(n^{2.376\cdots})$.

Quick Sort

Pivot Selection Methods

1. First Element

- Simple to implement
- Leads to $O(n^2)$ on sorted arrays

2. Last Element

- Also simple
- Same worst-case issues

3. Random Element

- Avoids worst-case on specific inputs
- Expected time: $O(n \log n)$

MIT Vishwaprayag University

6/6

Median-of-Three Strategy

Median-of-Three Pivot Selection

1. Choose first, middle, and last elements
2. Select the median of these three
3. Swap median with appropriate position

Example

Array: [5, 3, 8, 4, 2, 7, 1]
First: 5, Middle: 4, Last: 1
Sorted: [1, 4, 5] $\rightarrow$ Median $= 4$
Pivot $= 4$

- Reduces chance of worst-case behavior
- Better performance on partially sorted arrays

6/6

Median-of-Medians Strategy

Guaranteed $O(n \log n)$ Time

1. Divide array into groups of 5 elements
2. Find median of each group
3. Recursively find median of medians
4. Use this as pivot

Properties

- Guarantees good partition (30-70 split)
- Time complexity: $O(n \log n)$ worst-case
- Higher constant factors make it slower in practice
- Used in theoretical algorithms

MIT Vishwaprayag University

6/6

Karatsuba's Algorithm

Karatsuba’s Algorithm

6.006 Review Session

MIT Vishwaprayag University

Problem Statement

- Given two n-digit long integers a and b in base r, find a × b.
- We've always assumed this is a constant time operation.
  - Makes life simpler.
  - Numbers are usually relatively small.
  - As we'll see, we can do multiplication relatively fast.

Naïve Algorithm

- Using the algorithm we all love and know (the one we were taught in grade school) will take

$$O(n^2)$$

- Would like to improve on this…

Divide and Conquer

- Let's try divide and conquer.
  - Divide each number into two halves.
    - $x = x_H r^{n/2} + x_L$
    - $y = y_H r^{n/2 +} y_L$
  - Then:

$$xy = (x_H r^{n/2} + x_L) y_H r^{n/2 +} y_L$$
$$= x_H y_H r^n + (x_H y_L + x_L y_H) r^{n/2} + x_L y_L$$

  - Runtime?
    - $T(n) = 4 T(n/2) + O(n)$
    - $T(n) = O(n^2)$

Karatsuba's Insight

- Instead of 4 subproblems, we only need 3 (with the help of clever insight).
- Three subproblems:
  - $a = x_H\, y_H$
  - $d = x_L\, y_L$
  - $e = (x_H + x_L)\,(y_H + y_L) - a - d$
- Then $xy = a\, r^n + e\, r^{n/2} + d$
- $T(n) = 3\, T(n/2) + O(n)$
- $T(n) = O(n^{\log 3}) = O(n^{1.584\ldots})$

MIT Vishwaprayag University

Worked Example

- Compute 1234 * 4321.
- Subproblems:
  - $a_1 = 12 * 43$
  - $d_1 = 34 * 21$
  - $e_1 = (12 + 34) * (43 + 21) - a_1 - d_1$
    $= 46 * 64 - a_1 - d_1$
  - Need to recurse…

Worked Example

- First subproblem:

$$a_1 = 12 * 43$$

- Subproblems:
  - $a_2 = 1 * 4 = 4$
  - $d_2 = 2 * 3 = 6$
  - $e_2 = (1+2)(4+3) - a2 - d2$
    $= 11$
- Answer: $4 * 10^2 + 11 * 10 + 6 = 516$

Worked Example

- Second subproblem

$$d_1 = 34 * 21$$

- Subproblems:
  - $a_2 = 3 * 2 = 6$
  - $d_2 = 4 * 1 = 4$
  - $e_2 = (3+4)(2+1) - a2 - d2$
    $= 11$
- Answer: $6 * 10^2 + 11 * 10 + 4 = 714$

Worked Example

- Third subproblem:

$$e_1 = 46 * 64 - a_1 - d_1$$

- Subproblems:
  - $a_2 = 4 * 6 = 24$
  - $d_2 = 6 * 4 = 24$
  - $e_2 = (4+6)(6+4) - a2 - d2$
    $= 52$
- Answer: $24 * 10^2 + 52 * 10 + 24 - 714 - 516$
  $= 1714$

MIT Vishwaprayag University

Worked Example

- Final Answer:

$$1234 * 4321 = 516 * 10^4 + 1714 * 10^2 + 714$$

$$= 5{,}332{,}114$$